

LangChain RAG实践

主讲人：龙鹏（笔名言有三）

目录



- LangChain介绍与模块详解
- LangChain RAG应用实战

LangChain介绍与模块详解

LangChain



- 一个用于开发基于语言模型的应用程序的开发框架

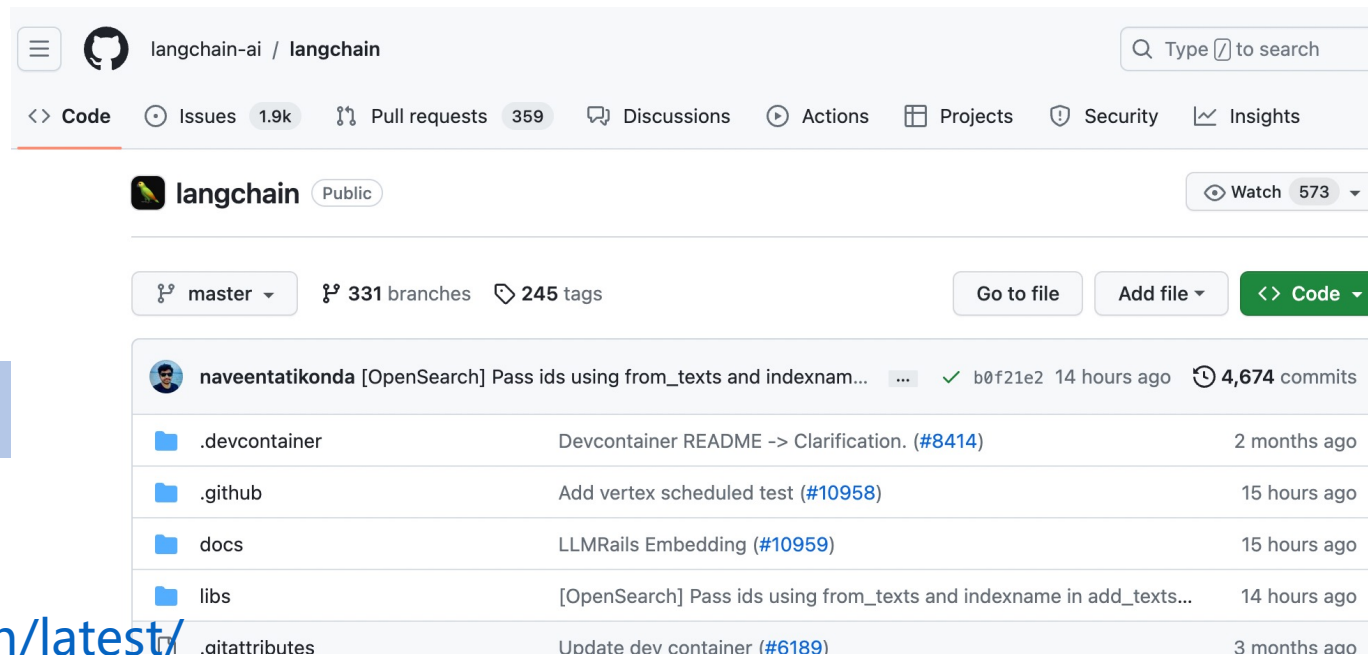


LangChain

安装方法: `pip install langchain`

英文社区: <https://python.langchain.com/en/latest/>

中文社区: <https://www.langchain.cn/>



<https://github.com/langchain-ai/langchain>

- LangChain的特性

- ① **中间件**: LangChain是一个链接面向用户程序和LLM之间的一个中间层, 支持各种模型类型和模型集成。
- ② **受青睐**: 它在 2023 年 3 月获得了 Benchmark Capital 的 1000 万美元种子轮融资, 后又拿到了红杉2000-2500万美金的融资, 估值已经提升到了2亿美金左右。
- ③ **端到端**: LangChain 可以轻松管理与语言模型的交互, 将多个组件链接在一起, 并集成额外的资源, 例如 API 和数据库。其组件包括了模型 (各类LLM), 提示模板 (Prompts), 索引, 代理 (Agent), 记忆等等。
- ④ **COT鼻祖**: Auto-GPT和BabyAGI所使用的链式思考能力都是由LangChain启发而来。

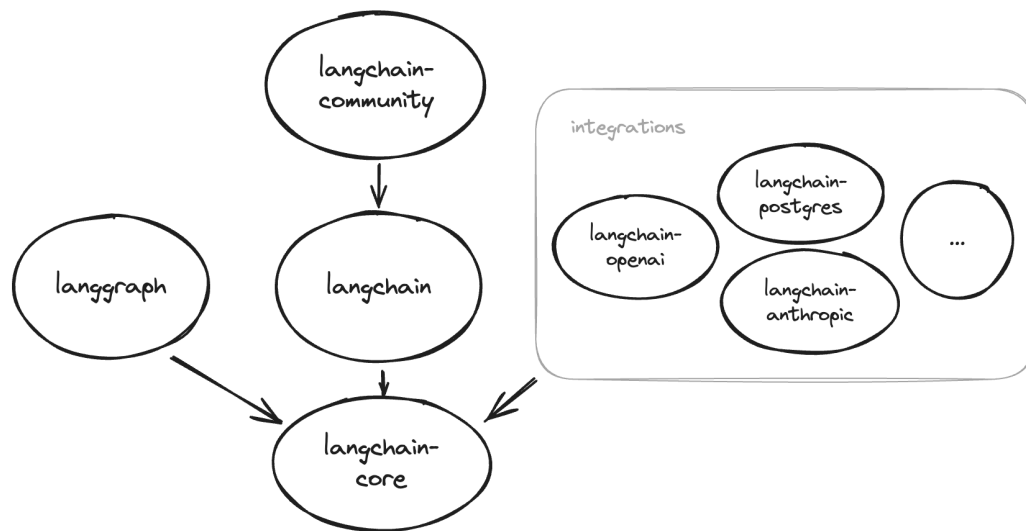
- LangChain的相关概念

- ① **Models**: langchain使用的LLM（大语言模型），是langchain应用的大脑
- ② **Prompts**: 用于为LLM构建Prompt，通常以promptTemplate的形式组织
- ③ **Memory**: 用于维护和构建用户在跟大模型交互过程中的上下文记忆
- ④ **Indexes**: 用于检索，对于待检索的文档，基于该工具可以进行embedding并检索
- ⑤ **Chains**: 最核心的概念，每个chain就是一个任务链，基于它可以完成一个完整的任务。langchain定义了各种chain，每种chain可以执行某种特定的任务，例如文档问答chain、保险销售chain等。
- ⑥ **Agents**: 用来管理chain，即何时调用何种chain，何种LLM。基于agent可以构建更强大的智能体。
- ⑦ **Callbacks**: 回调函数，负责调用外部API，获取实时数据

LangChain模块



- 整体项目结构



- ① langchain-core
- ② langchain
- ③ langchain-community
- ④ LangGraph
- ⑤ LangServe
- ⑥ LangSmith

最低安装依赖

```
pip install langchain #会自动安装langchain-core  
pip install langchain-openai  
pip install langchain-community
```

<https://python.langchain.com/v0.2/docs/introduction/>

LangChain模块

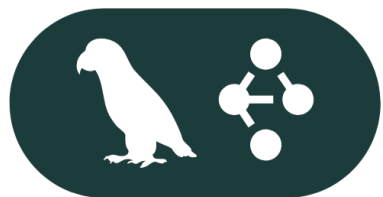


- langchain-core包、langchain包、langchain-community包
 - ① langchain-core: langchain的核心实现，包含了不同组件的基本抽象和组合方式。核心组件（如 LLMs、向量存储、检索器等）的接口都在这里定义。此包不包含任何第三方集成，依赖项被有意保持得非常轻量。
 - ② langchain: 包含了组成应用程序认知架构的chains, agents和retrieval strategies(检索策略)。所有的chains, agents和retrieval strategies(检索策略)都不是针对某一特定集成，而是适用于所有集成的通用功能。
 - ③ langchain-community: 包含了由 LangChain 社区维护的第三方集成。关键合作伙伴的包被单独分离出来。此package(包)包含了各种组件的所有集成（如 LLMs、向量存储、检索器）。为了保持包的轻量化，此包中的所有依赖项都是可选的。虽然大多数集成都在 langchain-community 包中，但仍将一些流行的集成分离成它们自己的包（例如 langchain-openai、langchain-anthropic 等）。

LangChain模块



- LangGraph, 基于 LangChain 构建的一种状态机驱动的智能体编排框架。



LangGraph

<https://langchain-ai.github.io/langgraph/>

通过图的形式构造LLM应用，旨在通过将步骤建模为图中的边和节点，将 Agent 系统建模设计为(有向图)流程图

特性	LangChain (Chain)	LangGraph (Graph)
控制流程	线性	可分支、循环、并行
复杂度管理	较难	模块化、清晰可控
多 Agent 协作	较难实现	易于设计多个节点交互
适合任务类型	单步/固定任务	多步骤、有状态流程
智能体开发	不适合 ❌	适合 ✅

- ✅ 更清晰的任务流程建模能力 ([流程图](#))
- ✅ 支持复杂控制流 (如循环、条件分支、并行)
- ✅ 状态可持久化，适合长周期任务
- ✅ 灵活集成 Tool、Memory、Agent 等组件



- ```
PS E:\dev\langchain\langserve-app> poetry run langchain serve --port 8080
INFO: Will watch for changes in these directories: ['E:\\dev\\langchain\\langserve-app']
INFO: Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
INFO: Started reloader process [35296] using StatReload
INFO: Started server process [44000]
INFO: Waiting for application startup.

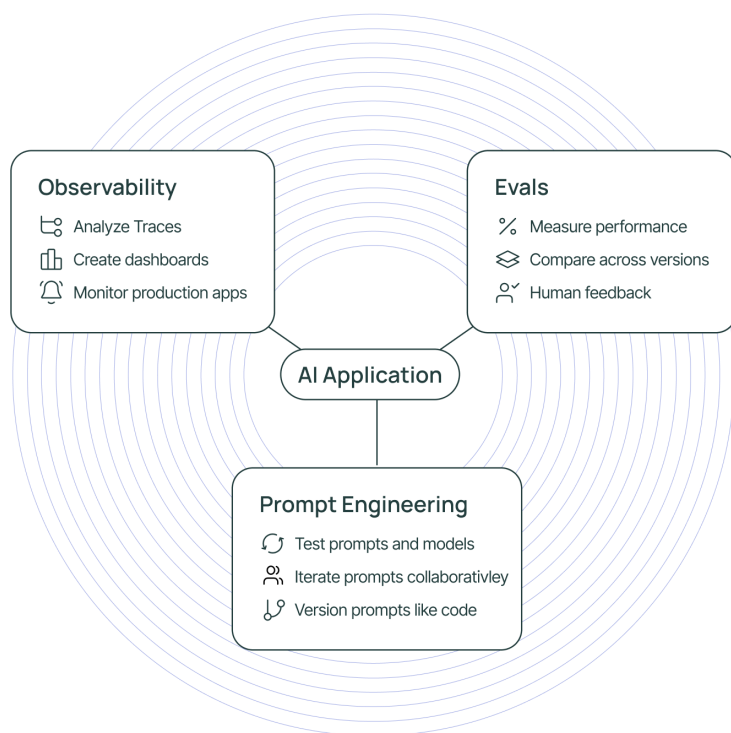
LANGSERVE
```
- LANGSERVE: Playground for chain "/" is live at:  
LANGSERVE: |  
LANGSERVE: └─> /playground/  
LANGSERVE:  
LANGSERVE: See all available routes at /docs/  
INFO: Application startup complete.

<https://python.langchain.com/v0.2/docs/langserve/>

# LangChain模块



- LangSmith: 一个调试、测试、评估和监控LLM 应用的开发者平台。

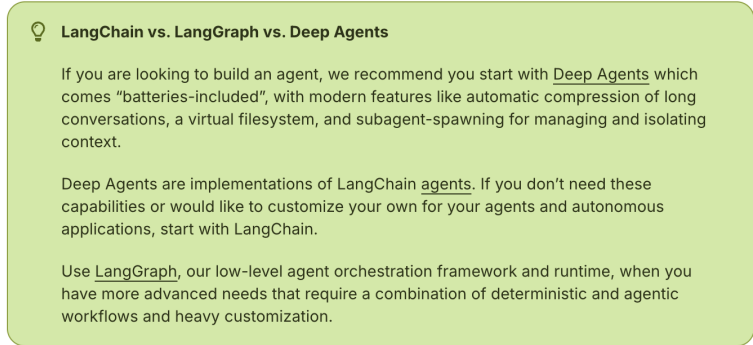


- Tracing (追踪) 功能: LangSmith 可以记录 LLM 应用的每次运行 (Run), 包括输入、输出、中间步骤、工具调用、API 请求等详细信息。
- 数据集管理功能: 允许开发者创建、存储和管理测试数据集, 用于评估和优化 LLM 应用。
- 评估 (Evaluation) 功能: 提供工具来评估 LLM 应用的性能, 支持自动化和手动评估。
- 监控 (Monitoring) 功能: 在生产环境中实时监控 LLM 应用的性能和行为。
- 协作与版本控制功能: 支持团队协作, 提供项目管理和版本控制功能。
- 集成与扩展功能: LangSmith 无缝集成 LangChain, 并支持其他 LLM 框架和工具。

<https://docs.smith.langchain.com/>



- <https://docs.langchain.com/oss/python/langchain/overview>

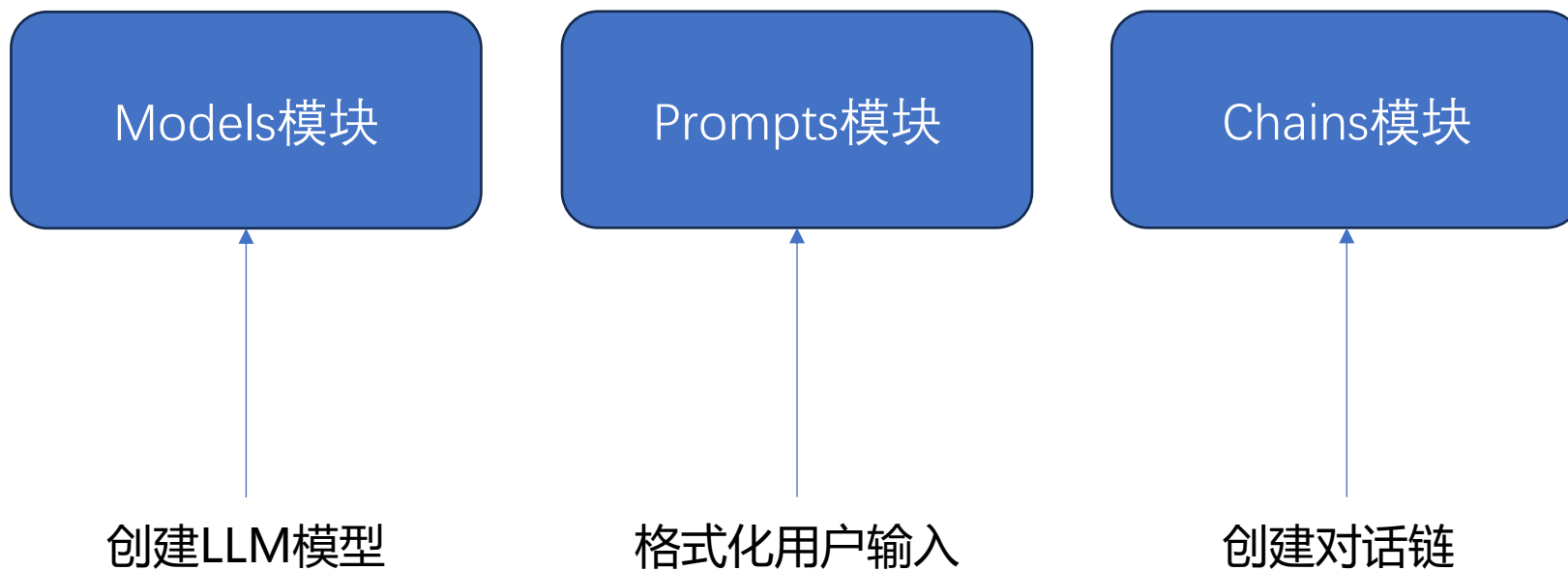


# LangChain RAG应用实战

# LangChain实战1-多轮对话系统



- 基于DeepSeek R1模型的多轮对话系统创建



# LangChain实战1-多轮对话系统



- Models模块, LangChain 不托管任何 LLM, 而是依赖于第三方集成, 可使用 langchain-openai等第三方库, 通过API调用Deepseek模型

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(
 api_key="sk-xxxxx",
 base_url="https://api.deepseek.com/v1",
 model="deepseek-chat")
```

- 也可以通过环境变量来设置key
- 也可以使用langchain\_deepseek来构建模型

# LangChain实战1-多轮对话系统



- Prompts模块，用于创建消息模板，包括系统消息/用户消息/聊天提示模板

```
from langchain.prompts import ChatPromptTemplate, SystemMessagePromptTemplate,
HumanMessagePromptTemplate

定义系统消息模板
system_template = "你是一个多轮对话助手."
system_message_prompt = SystemMessagePromptTemplate.from_template(system_template)

定义用户消息模板
human_template = "{text}"
human_message_prompt = HumanMessagePromptTemplate.from_template(human_template)

构建对话提示模板
chat_prompt = ChatPromptTemplate.from_messages([system_message_prompt, human_message_prompt])
```

SystemMessage表示一个角色为“系统”的消息，告诉模型如何行为。并不是所有模型提供者都支持此功能。  
HumanMessage将根据用户传入的输入变量进行格式化。  
ChatPromptTemplate聊天提示模板用于格式化消息列表。由多个子模板组成。



# LangChain实战1-多轮对话系统

- chains模块，通常将大语言模型（LLM）与提示（Prompt）结合在一起，可以对文本或数据进行一系列操作。

```
from langchain.chains import LLMChain
chain = LLMChain(llm=llm, prompt=chat_prompt)
```

链接受用户输入，使用提示模板对其进行格式化，然后将格式化的响应传递给 LLM 。我们可以通过将多个链组合在一起，或者通过将链与其他组件组合在一起构建更复杂的链。

# LangChain实战1-多轮对话系统



- 代码运行效果

```
MacBook-Pro-4:deepseek-langchain longpeng$ python3 chat_deepseek.py
```

```
欢迎使用 Deepseek 多轮对话系统！输入 'exit' 结束对话。
```

```
你：你好，先认识一下吧
```

```
Deepseek：你好呀！很高兴认识你～我是DeepSeek Chat，一个由深度求索公司打造的AI助手，可以陪你聊天、解答问题、提供各种帮助。✨
```

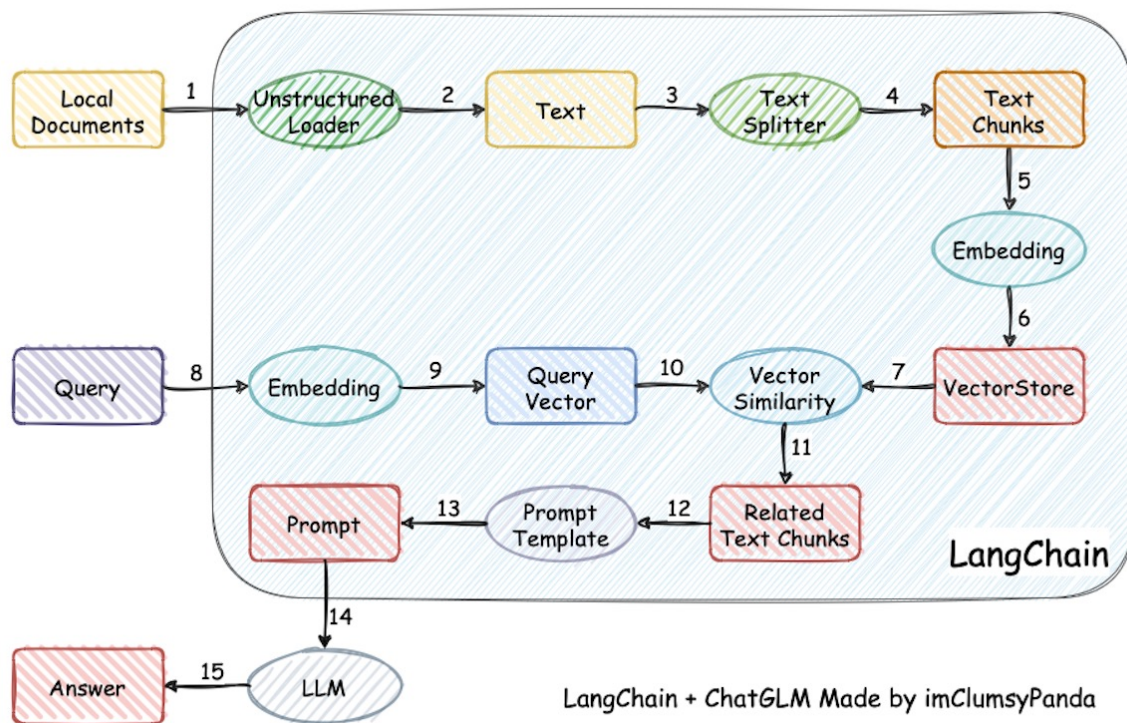
```
你可以叫我DeepSeek或者小深～我擅长知识问答、写作辅助、学习指导，还能处理上传的文档。最近刚升级到**DeepSeek-V3版本**，知识更新到2024年7月，支持**128K超长上下文**，而且**
完全免费**！😊
```

```
你呢？怎么称呼你呀？或者有什么想聊的话题吗？
```

```
你：█
```

# LangChain实战2-文本向量化

- 基于Huggingface嵌入模型的本地多源数据加载与文本向量化



- (1-2) 准备本地知识库文档，目前支持 txt、docx、md、pdf 格式文件，使用Unstructured Loader类加载文件，获取文本信息
- (3-4) 对文本进行分割，将大量文本信息切分为chunks
- (5) 选择一种embedding算法，对文本向量化
- (6) 将embedding结果保存到数据库

# LangChain实战2-文本向量化

- 需要使用到的核心组件
  - 步骤1：文本进行索引：加载文件，分割文本，进行索引。
  - 步骤2：文本嵌入：使用OpenAIEmbeddings等进行嵌入。
  - 步骤3：存储数据：将得到的向量保存在数据库。

| 组件                                     | 类型   | 子类型    |
|----------------------------------------|------|--------|
| UnstructuredFileLoader等                | 索引组件 | 文本加载器  |
| RecursiveCharacterTextSplitter         | 索引组件 | 文本分割器  |
| OpenAIEmbeddings/HuggingFaceEmbeddings | 模型组件 | 文本嵌入模型 |
| Chroma                                 | 索引组件 | 向量库    |

# LangChain实战2-文本向量化



- 文档加载支持的格式类型

txt

csv

html

json

markdown

pdf

docx,  
xlsx,pptx

本地目录

需要加载的相关库

```
from langchain_community.document_loaders import UnstructuredFileLoader
from langchain_community.document_loaders.csv_loader import CSVLoader
from langchain_community.document_loaders import UnstructuredHTMLLoader
from langchain_community.document_loaders import JSONLoader
from langchain_community.document_loaders import UnstructuredMarkdownLoader
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.document_loaders import AzureAIDocumentIntelligenceLoader # 需要API

from langchain_community.document_loaders import DirectoryLoader
```

# LangChain实战2-文本向量化



- 文档分割

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=0)
chunk_size为每个分片的最大大小, chunk_overlap分片之间的覆盖大小
split_docs = text_splitter.split_documents(data)
```

默认的分割符为[ "\n\n" , "\n" , " " , "" ] , 中文不一定合适, 可以重写

```
text_splitter = RecursiveCharacterTextSplitter(
 separators=[
 "\n\n",
 "\n",
 " ",
 ".",
 ",",
 "\u200b", # Zero-width space
 "\uff0c", # Fullwidth comma
 "\u3001", # Ideographic comma
 "\uff0e", # Fullwidth full stop
 "\u3002", # Ideographic full stop
 "",
],
 # Existing args
)
```

# LangChain实战2-文本向量化



- 载入文档、切分

```
MacBook-Pro-4:deepseek-langchain longpeng$ python3 data_loader.py
文件页数与内容预览 98 <class 'langchain_core.documents.base.Document'> page_content='人生能有几个十年
高中那些事儿
清箫居士' metadata={'producer': '', 'creator': 'WPS Office', 'creationdate': '2016-03-25T23:47:58+15:47', 'author': '新邵二中
-03-25T23:47:58+15:47', 'sourcemodified': "D:20160325234758+15'47'", 'subject': '回忆录', 'title': '高中那些事儿', 'trapped':
: 0, 'page_label': '1'}
切割后文档尺寸: 849 <class 'list'>
```

# LangChain实战2-文本向量化



- 目前有两种常见嵌入方案

## OpenAIEmbeddings

- 使用简单，并且效果比较好；
- 会消耗openai的token，特别是大段文本时，消耗的token还不少，如果知识库是比较固定的，可以考虑将每次生成的embedding做持久化，这样就不需要再调用openai了，可以大大节约token的消耗；
- 可能会有数据泄露的风险，如果是一些高度私密的数据，不建议直接调用。

## HuggingFaceEmbeddings

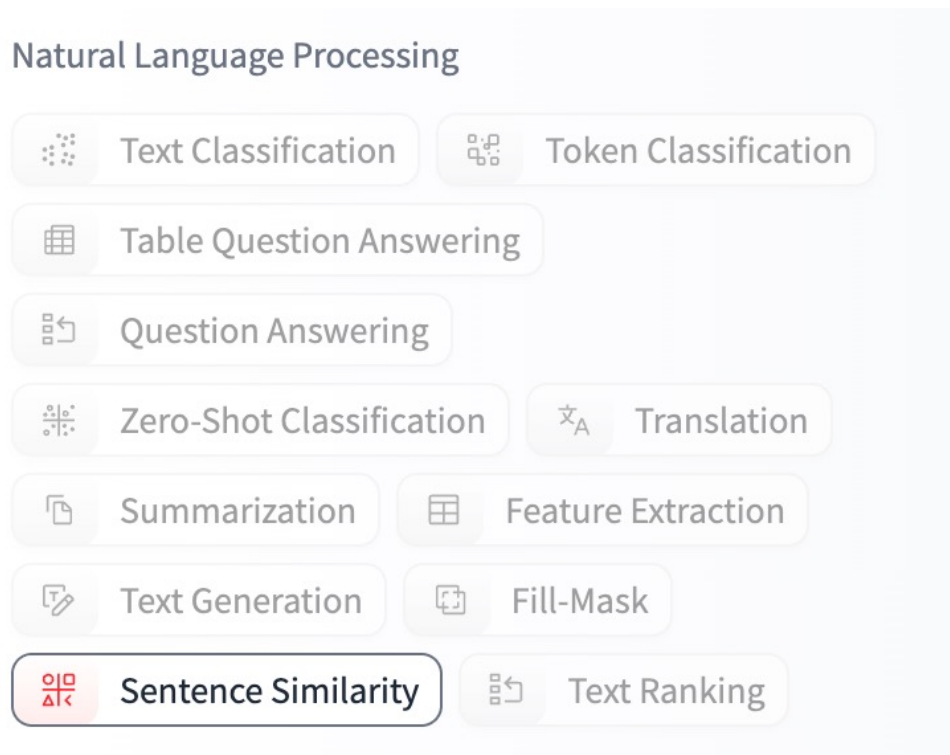
- 可以用HuggingFace模型来进行实验，数据都是在本机上进行计算
- 需要一定的硬件支持，最好是有GPU支持，不然生成数据可能会非常慢
- 生成的向量效果可能不是很好，HuggingFace上的中文向量模型不是很多。



# LangChain实战2-文本向量化



- Huggingface嵌入模型配置



|  |                                                 |      |                     |       |     |
|--|-------------------------------------------------|------|---------------------|-------|-----|
|  | Sentence Similarity                             | 0.6B | Updated Aug 9, 2024 | 2.01M | 59  |
|  | Alibaba-NLP/gte-large-en-v1.5                   | 0.4B | Updated Aug 9, 2024 | 1.91M | 221 |
|  | sentence-transformers/multi-qa-MiniLM-L6-cos-v1 | 0.0B | Updated Nov 6, 2024 | 1.88M | 133 |
|  | nomic-ai/nomic-embed-text-v1.5                  | 0.1B | Updated 16 days ago | 1.75M | 680 |

去Huggingface模型库中选择模型，根据sentence similarity或者feature extraction进行过滤

# LangChain实战2-文本向量化



- 多种不同的调用方式，从Inference Providers中获取

GanymedeNil / **text2vec-large-chinese** like 754

Sentence Similarity Transformers PyTorch Safetensors Chinese bert feature-extraction text2vec text-embeddings-inference License: apache-2.0

Model card Files and versions Community 24

Based on the derivative model of <https://huggingface.co/shibing624/text2vec-base-chinese>, replace MacBERT with LERT, and keep other training conditions unchanged.

News

2024-06-25 [text2vec-large-chinese](#) onnxruntime version.

Talk to me: <https://twitter.com/GanymedeNil>

Downloads last month: 199,234

Safetensors Model size: 326M params Tensor type: I64 · F32 Files info

Inference Providers **NEW** HF Inference API

Sentence Similarity Examples

Source Sentence: 今天天气不错

Sentences to compare to: 昨天下了大雨, 你吃了吗

Add Sentence

Compute

昨天下了大雨 0.399

你吃了吗 0.126

View Code Snippets 32.4s Cached example Maximize

HF Inference API在线推理试用，Hugging Face 官方部署提供算力。

# LangChain实战2-文本向量化



- 将模型下载到本地，可加快速度

|                             |                  |            |                                                 |                   |                    |              |
|-----------------------------|------------------|------------|-------------------------------------------------|-------------------|--------------------|--------------|
| main text2vec-large-chinese |                  | Go to file | K                                               | 3 contributors    | History: 9 commits | + Contribute |
| GanymedeNil                 | Update README.md | add4e02    | VERIFIED                                        | about 1 year ago  |                    |              |
| .gitattributes              | Safe             | 1.48 kB    | initial commit                                  | over 2 years ago  |                    |              |
| README.md                   | Safe             | 490 Bytes  | Update README.md                                | about 1 year ago  |                    |              |
| config.json                 |                  | 821 Bytes  | Upload 6 files                                  | over 2 years ago  |                    |              |
| eval_results.txt            | Safe             | 69 Bytes   | Upload eval_results.txt                         | over 2 years ago  |                    |              |
| model.safetensors           | Safe             | 1.3 GB xet | Adding `safetensors` variant of this model (#9) | about 2 years ago |                    |              |
| pytorch_model.bin           | Safe pickle      | 1.3 GB xet | Upload 6 files                                  | over 2 years ago  |                    |              |
| special_tokens_map.json     | Safe             | 125 Bytes  | Upload 6 files                                  | over 2 years ago  |                    |              |
| tokenizer.json              | Safe             | 439 kB     | Upload 6 files                                  | over 2 years ago  |                    |              |
| tokenizer_config.json       |                  | 514 Bytes  | Upload 6 files                                  | over 2 years ago  |                    |              |
| vocab.txt                   | Safe             | 110 kB     | Upload 6 files                                  | over 2 years ago  |                    |              |

<https://huggingface.co/GanymedeNil/text2vec-large-chinese/tree/main>

# LangChain实战2-文本向量化

- Huggingface嵌入模型配置

```
from langchain_huggingface import HuggingFaceEmbeddings
EMBEDDING_MODEL = "text2vec"
embedding_model_dict = {
 "text2vec": "embedding_models/text2vec-large-chinese" # 使用本地目录
}
embeddings = HuggingFaceEmbeddings(model_name=embedding_model_dict[EMBEDDING_MODEL],
model_kwargs={"device": "cpu", "trust_remote_code": True}
encode_kwargs={"normalize_embeddings": True, "batch_size": 32}
multi_process="False",show_progress="True")
```

model\_name参数: 指定 Hugging Face 上的预训练模型名称或本地路径

model\_kwargs: 用于控制模型加载行为, 常见选项入device: 指定设备 (如 "cuda", "cpu")

encode\_kwargs: 控制文本编码 (推理) 过程

multi\_process: 是否启用多进程加速

show\_progress:是否显示嵌入生成的进度条

# LangChain实战2-文本向量化



- 使用Chroma库在本地存储知识

```
#----- 第四步：存储结果 -----#
from langchain.vectorstores import Chroma
db = Chroma.from_documents(split_docs, embeddings, persist_directory="./data/doc_embedding")
print("data saved")
```



chroma.sqlite3



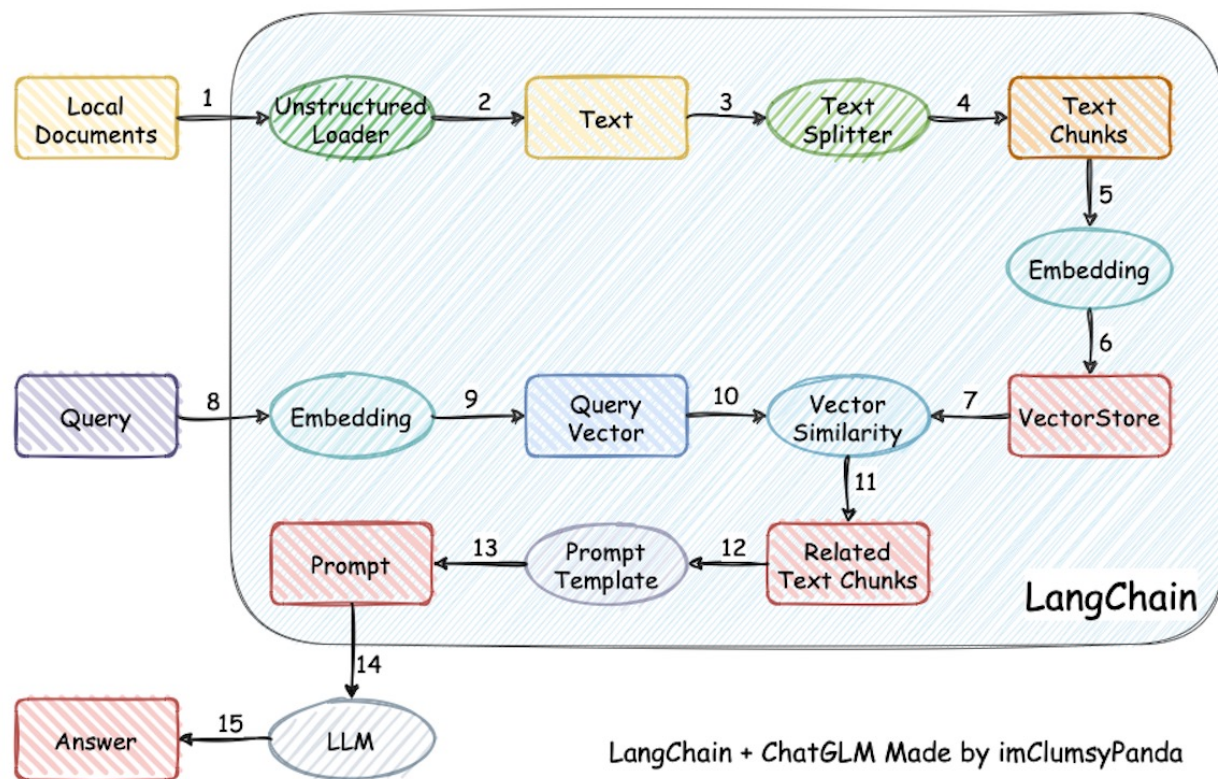
f072c0d3-  
bc3a-4...84b4721



# LangChain实战3-问答系统搭建



- 基于Chroma与PyQT的本地知识库检索问答系统搭建



- (1-2) 准备本地知识库文档，目前支持 txt、docx、md、pdf 格式文件，使用Unstructured Loader类加载文件，获取文本信息
- (3-4) 对文本进行分割，将大量文本信息切分为chunks
- (5) 选择一种embedding算法，对文本向量化
- (6) 将知识库得到的embedding结果保存到数据库
- (8-9) 将问题也用同样的embedding算法，对问题向量化
- (10) 从数据库中查找和问题向量最相似的N个文本信息
- (11) 得到和问题相关的上下文文本信息
- (12) 获取提示模板
- (13) 得到输入大模型的prompt，比如：问题，通过以下信息汇总得到答案。
- (14) 将prompt输入到LLM得到答案

# LangChain实战3-问答系统搭建



- 需要使用到的核心组件

| 组件                                     | 类型   | 子类型    |
|----------------------------------------|------|--------|
| OpenAIEmbeddings/HuggingFaceEmbeddings | 模型组件 | 文本嵌入模型 |
| ChatOpenAI                             | 模型组件 | LLM    |
| RetrievalQA                            | 检索组件 | 检索器    |

- 检索器：检索器可以搜索数据集，查询相关文本片段。
- 对话链：使用链来调度模型和检索器，接收理解用户输入，检索文本片段后生成答案。

# LangChain实战3-问答系统搭建



- 使用检索器来构建文档问答能力

```
db = Chroma(persist_directory="data/doc_embedding", embedding_function=embeddings)
retriever = db.as_retriever()
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(
 temperature=0.95,
 model="deepseek-chat")

qa = RetrievalQA.from_chain_type(llm=llm, chain_type="stuff", retriever=retriever)
```



# LangChain实战3-问答系统搭建



- 使用PyQT进行界面封装

1. 支持本地选择数据库
2. 支持问题输入与结果展示

基于 DeepSeek 的知识库检索问答系统

数据目录: /Users/longpeng/Desktop/2025年/1-实战营/deepseek-langchain/data/doc\_embedding 选择目录

文本输入:

新邵二中是什么|

检索

检索结果:

{'query': '新邵二中是什么', 'result': '根据提供的上下文，新邵二中是一所高中学校，位于中国湖南省娄底市新邵县。文中提到作者曾在新邵二中就读高三，并提到该校与其他学校（如冷水江一中）存在生源竞争关系。该校有237班等班级，且校长和老师（如化学老师陈春保）对优秀学生较为关注。此外，新邵二中会参与县级联考（如新邵县联考），与其他学校进行学业比拼。\\n\\n但上下文未提供更具体的学校背景信息（如建校时间、详细地址等）。若需进一步了解，可能需要查阅当地教育资料或学校官方信息。'}

# LangChain实战-综合思考



- 如何改进检索性能

文本分割方式

LLM模型

嵌入模型

嵌入模型参数

1. 检验大模型是否能够准确的理解用户问题?
2. 检验大模型是否能够准确的提炼文档的信息, 并给出准确的答案?
3. 检验大模型是否具备一定的推理能力?

扫一扫，关注有三AI公众号



三人行，必有AI